

# ChangeLog

---

## 1.1.2

- new feature - glitches. Please read the documentation for tips on how to configure this new feature

## 1.1.1

- support for **Volume Framework** - new custom CRTVolumeComponent with all CRT Filter attributes
- added legacy (CG) versions of shaders for Unity 2022.x or Unity 6 in compatibility mode

## 1.1.0

- updated for the **new Unity 6 render pipeline** and its Core RP Library 17.0.3 with IRenderGraphRecorder
- updated blipping, adjusted shader, using proper texture handles
- **new shader with limited features to improve performance on mobile devices** (see 'Improving performance' chapter below)
- adjusted ranges for various parameters

## 1.0.5

- new feature - shadowlines can now be set to negative values to change orientation from horizontal to vertical

## 1.0.4

- updated for URP 14.0.8
- updated for Unity 2022.3.8f1
- removed unused libraries that could cause issues with the latest Unity versions
- fixed "upside-down" camera render behavior during edit time and pause
- added injection point to parameters for better control when used outside full-screen pixel-perfect setups
- removed unnecessary parameter passes for URP 14.0.8 (cleaner code)

## 1.0.3

- updated for URP 14.0.7
- use of RTHandles instead of Textures

# Overview

---

Customizable CRT effect usable either as a full-screen effect or on any camera in a URP project. It uses a high-performance single-pass shader to create many CRT "artifacts". This shader can also be used separately in advanced scenarios.

**Please read Installation instructions below to properly configure the filter.**

Thanks for using my CRT Filter for URP.

## Package contents:

---

This package includes the following assets:

- SampleScene folder - sample scene and required assets (screenshot from the game Cursed Castilla - credits to Locomalito, Gryzor87, Abylight Studios)  
Note: the sample scene will not work properly until your URP pipeline is configured. See Installation instructions below.
- **Scripts\CRTRendererFeature** - main script for the ScriptableRendererFeature
- **Scripts\CRTVolumeComponent** - custom CRT Volume Component to be used with Unity's Volume Framework
- Scripts\ReadMeInfo - helper script in the sample scene pointing to this documentation
- **Shaders\CRTFilter** - main shader for the filter
- **Shaders\CRTFilterLite** - reduced-feature shader optimized for mobile devices
- Shaders\CRTFilterUnityCG - UnityCG version for **Unity 2022.x.x**
- Shaders\CRTFilterUnityCGLite - UnityCG version of the lite shader for **Unity 2022.x.x**
- Settings\ExampleCRTFilter2DRenderer - example renderer settings (see Installation instructions)
- Settings\ExampleUniversalRPSettings - example URP settings (see Installation instructions)

## Installation instructions for full-screen CRT effect using Pixel Perfect Camera:

### 1. Create a new 2D Renderer or adjust the provided ExampleCRTFilter2DRenderer.

You can create a new 2D Renderer via Right-click -> Create -> Rendering -> URP 2D Renderer, or use/rename/relocate the provided ExampleCRTFilter2DRenderer in the Settings folder.

### 2. Locate your URP settings

This package works only with URP. Your URP settings asset is usually located in the root Assets

folder or in Settings,

commonly named UniversalRP. If you cannot find it:

- Open Project Settings -> Graphics.
- The URP settings file is shown in **Default Render Pipeline** (or in Unity 2022: Scriptable Render Pipeline Settings).
- Clicking it will highlight the file in the Project window.

### 3. Configure URP settings

Note: URP uses renderer passes for both Game and Scene views. If you change the default renderer (first item in the Renderer

List), the CRT filter will also affect the Scene view. A better approach is to add a new renderer used only by selected cameras.

Open the URP settings file and:

- scroll to Renderer List
- click + to add a new renderer
- assign the renderer created in step 1 to this new slot

### 4. Camera configuration (required for the sample scene)

Once the renderer is added to the URP Renderer List:

- select the camera you want to apply the CRT filter to
- open the Camera component -> Rendering section
- choose the renderer created in step 1 in the Renderer dropdown

### 5. Configure the 2D Renderer

Note: If you use the provided ExampleCRTFilter2DRenderer, it already contains the CRT Renderer Feature. You may skip this step.

- open the 2D Renderer from step 1
- scroll to Renderer Features
- click Add Renderer Feature -> choose CRT Renderer Feature
- expand the feature
- rename it (recommended: "CRT Filter") to avoid known naming issues in older Unity editors
- assign the CRTFilter shader
- assign the CRTFilterLite shader (optional - you need lite version only if used on mobile device)

### 6. Configure the CRT Filter

CRT Filter settings live inside the 2D Renderer. You may create multiple 2D Renderers with different CRT configurations

(useful for example for in-game monitors).

- open the 2D Renderer
- expand CRT Renderer Feature
- choose any preset to preview the effect
- or choose **custom** preset to adjust parameters (see Reference below)
- **IMPORTANT:**

For pixel-based effects (blur, smudge, bleed, and partially scanlines), you must set the

correct pixel resolution (X and Y).

The filter processes the texture as if it had this pixel resolution, regardless of actual screen resolution. This is essential

for retro/pixel-art games (e.g., a 320x240 game rendered at 4x scale). The CRT filter must know the logical pixel resolution to avoid artifacts.

## 7. Custom color spaces

If you use a non-standard color space, adjust the temporary render color buffer format in `CRTRendererFeature.cs` -> `OnCameraSetup`.

# Configuration of glitches

Most parameters of the CRT Filter are self-explanatory and can be configured by using sliders to achieve desired effect. Glitches

are similar, but there are some specific combination of parameters to achieve particular glitches.

## Glitch parameters:

- **Glitch Frequency** how often a glitch appears. 1 = glitch always visible, 0 = no glitches.
- **Glitch Bands** splits the screen into the specified number of horizontal bands. This indirectly defines each band's height.  
0 = no glitches, 1 = full-screen glitch, n = number of bands.
- **Glitch Position** when more than one band is used, this selects which band is currently "active".  
Irrelevant if **Movement Speed** is greater than 0.
- **Glitch Position Flicker** adds small, random vertical jumps to the active band.
- **Glitch Movement Speed** if non-zero, the active band slowly (or quickly) moves from the top of the screen to the bottom.
- **Glitch Strength** controls the horizontal pixel displacement inside the glitch band, or the intensity/color of "white noise" when Noise is above value 1.
- **Glitch Noise** - defines the visual style of the glitch. 0 = smooth horizontal pixel shift. Approaching value 1, the band is split into sub-bands (1-pixel-high) that shift randomly. At 1, smooth shift is fully replaced by random shift. From 1 to 2, white noise appears, starting as small dots and becoming full lines at value 2.  
Note: sub-band corresponds to a pixel size defined by **Pixel Resolution Y** parameter
- **Glitch Noise Speed** speed of the random flicker within each sub-band.

## Particular glitches

- **VHS noise** – for VHS white noise, **Glitch Noise** parameter should be set above value 1 and **Strenght**, **Bands** and **Position** parameters set for preffered color, size and position of the noise. **Position Flicker** should be above zero to cause small flickering of the band, but **Movement Speed** should stay at 0, as VHS glitches are usually in the constant place due to missalignement of the player head. Similarly **Frequency** should stay at 1 as VHS glitches doesn't disapper suddently. If the smooth appearance and dissapearance of the effect is required, either **Bands** parameter can be animated to slowly decrease the size of the band until is just 1 pixel and then the effect can be disabled. This will cause smooth dissapearance of the effect but it's not supported directly by the filter and parameter has to be animated in the custom code.
- **Random glitches** – main parameter needed for random glitches is **Frequency**. It should be set to lower values, even very low like 0.001f. Then it depends how the glitch should look like. few examples:
  - fullscreen sudden appear of white noise - bands=1, noise>1
  - small strike randomly appearing on the screen - bands between 5 and 100, movement speed > 7 to achieve random appearance, noise>1
  - small horizontal shift - similar to previous one, but instead of white noise, screen is just shifted horziontally - bands between 5 and 20, movement speed > 7 to achieve random apperance, noise < 1, strength between 0.05 and 0.4
- **Moving glitch** – this is a behaviour of some old CRTs with terestrial signal. Small band of shifted line is moving on the screen. The shift can be either quite visible or very subtle. **Frequency** = 1 to keep the glitch visible all the time, **Bands** for prefered height of the glitch, **Position** is irrelevant, **Position Flicker** can be set above 0 to add small shakes of the moving band, **Movement Speed** for prefered speed of the glitch, **Noise** = 0 or very close to 0. This kind of glitch isn't random and is rather smooth, so noise should be minimal. **Strength** should be also rather low. It can be either very close to 0 for just subtle effect, or slightly higher to be more pronauced, but this kind of effect isn't realistic with strength above 0.01f.

## ADVANCED – Improving performance (mostly for mobile devices)

This CRTFilter uses a single-pass shader, so when a GPU is available the performance impact is negligible. On mobile devices, or any device without a proper GPU, the per-pixel math can reduce performance. The calculations themselves are already optimized and cannot be removed, but you can skip work for effects you don't use. Starting with version 1.1.0, the package includes a CRTFilterLite shader that can be selected in the configuration. You can enable it either by checking "Use Shader Lite" or by directly assigning CRTFilterLite in the Shader field. Enabling "Use Shader Lite" also disables unsupported sliders. (If slider changes do not affect the camera renderer, restart the scene so the pipeline can rebuild the material.)

Lite shader keeps the essential effects:

- vignette and screen geometry
  - bleed
  - scanlines, shadowlines, and noise
- and removes subtle, GPU-heavy effects:
- blur and smidge
  - aperture lines
  - glitches
  - image adjustments (brightness, contrast, gamma, etc.)

If you still don't use some of the remaining Lite effects, you can further improve performance by disabling their math directly in the shader.

Open the Lite shader and comment/uncomment the relevant lines in ApplyCRT around line 218.

## ADVANCED – Using Volumes

CRTFilter from version 1.1.1 supports volumes and provides CRTVolumeComponent. This component contains all variables of the CRT filter and may be used as any other VolumeControl with VolumeProfiles. Use of volumes is configured by parameter **Use Volume Component**. Once enabled, all attributes in the CRT Renderer Feature are ignored (can't be even modified as the profile is automatically set to 'None') and values from calculated volume are used.

If volumes doesn't work or CRT filter reports error - refer to the Troubleshooting section below.

Usage of VolumeControls and VolumeProfiles is out of the scope of this documentation - please refer to public Unity Volume Framework documentation.

## ADVANCED – Using CRT effect on other than main camera

You can use CRT filter on any camera in the project and each can have different CRT effect configuration. You may proceed similarly as in **installation instructions** above and just add proper 2DRenderer to any camera. If more cameras use the same renderer, they will share CRT filter settings obviously. If you need different settings for different camera(s), you need to create more 2DRenderers.

## ADVANCED – Using CRT filter on non pixel/retro game or camera

This filter can be used on any graphic style (even 3D), although it was created specifically for 2D pixel game using Pixel Perfect Camera.

Specific for 2D games are just pixel effects (blur, smidge, bleed and to some extent also scanlines).

Shader checks each VISUAL pixel

(as user sees them - e.g. 320x240) and apply math to effect each TEXTURE pixel (by texture resolution - e.g. 1920x1080). If the texture

is not "pixelated" and this virtual resolution (e.g. 320x240) is not properly set in CRT filter settings, results are not guaranteed and

based on graphical style may be better or worse. Try to play with settings to reach acceptable results.

## ADVANCED – Usage of shader

Main value of this CRT filter is its shader. If you know how, you may use it in your own pipelines to achieve nice effect in some

advanced scenarios. To describe use of shader is out of scope of this doc and also of the CRT filter package itself.

## Requirements

CRT filter requires URP to work properly. Shader can be used also in other pipelines, but provided script and

instructions are implemented for URP.

# Limitations

CRT filter can't be used without component that creates camera texture such as PixelPerfectCamera or postprocessor

on the camera. Any other component that renders to camera texture can be used, but the filter works best with

PixelPerfectCamera and with resolution set to same values in the CRT filter and PixelPerfectCamera component.

# Reference

CRT filter is configured in the 2DRenderer asset (see 'Installation instructions' above).

Most parameters are either self-explanatory or their effect can be easily seen if changed.

Some important notes:

- if preset is set to any value except "custom", most parameters can't be changed and are fixed by selected preset
- you may choose any preset and then change preset to "custom" to modify all values
- or you may choose preset "none" and then change it to "custom" to start from scratch
- Pixel Resolution X & Y has to be set to the same values as in PixelPerfectCamera component - otherwise some effects may be misaligned (see some explanations above)
- smidge effect is design to be used only with the bleed effect, without bleed, smidge doesn't look right
- to set custom offsets for R, G and B, Chromatic Aberration has to be set to 0
- see values used for presets for inspiration

# Unity 2022.x.x or Unity 6 Compatibility Mode (without Render Graph)

The latest version of this asset is adjusted to Unity 6 and it's new and improved rendering pipeline using Render

Graph. If you would like to use CRT filter with previous version of Unity or in Unity 6 with Compatibility Mode

enabled (without Render Graph), please download previous version of this asset (1.0.5) or adjust CRTRendererFeature.cs

class by commenting out existing RecordRenderGraph and Dispose methods and uncommenting section at the end and by

using UnityCG variants of both full and lite shaders. Instructions are provided directly in the CRTRendererFeature.cs class.

# Troubleshooting

---

- **values in CRT renderder feature can't be modified**

- values can be modified only if the profile is set to 'Custom'. Any other profile overwrites all values and prevents its modification
- if "Use Volume Control" option is enabled, 'None' profile is forced and values can't be modified. If this option is enabled, values in Renderer feature are ignored anyway and values from CRTVolumeComponent are used instead
- if you want to modify values of the preset, first select a preset (values will be changed and locked) and then switch to 'Custom' preset (values will stay as set by previous preset but are unlocked)

- **sample scene doesn't use CRT filter**

- if you've opened sample scene in your existing project, your URP settings still use standard renderer.  
You have to configure your URP settings (see 'settings' above) and properly set camera in the sample scene to use a renderer that you've properly configured

- **no output of the camera**

- disable CRT filter in the 2DRenderer to be sure, that camera is working properly without the filter
- check the console, if there isn't a warning regarding not existing camera texture. If there is the message, make sure there is some component that renders to camera texture (such as PixelPerfectCamera with crop enabled or standard camera with postprocessing enabled)
- try change any other value in 2DRenderer (for example disable and enable post-processing)

- **camera is 'upside-down' during the edit mode**

- there was a bug introduce with a specific combination of system + unity editor version + URP version.  
It was fixed in the version 1.0.4.

- **CRT effect is not visible**

- make sure, that camera is using proper renderer (camera settings in camera inspector)
- make sure, that renderer has added CRT Filter and it's enabled
- make sure, that filter has some settings (if everything is set to 0, there is no effect from the filter)

- **there is an error in the console**

- try to fix it by yourself - both renderer class and shader is available to you
- contact me: [curio124@gmail.com](mailto:curio124@gmail.com)
- **CRT filter decreases the performance of the game**
  - try to use Lite version of the shader -
  - see chapter 'ADVANCED - Improving performance (mostly for mobile devices)' above
- **CRT filter not working in Unity 6**
  - make sure, "Compatibility Mode" in Project Settings -> Graphics -> section "Render Graph" is disabled
  - if "Compatibility Mode" is required, follow instructions in chapter 'Unity 2022.x.x or Unity 6 Compatibility Mode' above
- **VolumeControl doesn't work or CRTFilter reports an error that CRTVolumeComponent can't be retrieved**
  - final volume with calculated values to be used for CRTFilter is retrieved from VolumeManager. Based on the specific settings of your project and/or usage of multiple cameras, there may be some issues connected with proper VolumeControl retrieval. Pls. adjust the code in the CRTRendererFeature.cs class in the AddRenderPasses and retrieve the volume based on your specific situation